SpatFD v2: estructura del paquete

Guía para desarrolladores

Documento de desarrollo

Octubre de 2026

Este documento describe cómo está construido SpatFD v2 por dentro: qué hay en cada archivo, cómo es el objeto `SpatFD`, por dónde pasa una predicción y qué hay que tocar para extender el paquete. Está pensado para quien vaya a mantenerlo. La teoría de cada método está en la documentación de las funciones (sección *Theory*) y el diseño del *solver* en `dev/diseno_solver.pdf`; aquí solo se repite lo necesario para entender el código.

Índice

1. Principios de diseño **2**

2. Flujo de trabajo **2**

3. Organización de los archivos **3**

4. El objeto `SpatFD` **3**
- 4.1. Estructura . . . 3
- 4.2. Invariantes . . . 4
- 4.3. Constructor, accesores y modificadores . . . 4
- 4.4. Construcción: `spatfd()` . . . 5
- 4.5. Combinación: `+.SpatFD` . . . 5

5. Intercambio con `gstat` **5**
- 5.1. `togstat()` . . . 5
- 5.2. `toSpatFD()`: el esquema tabular . . . 5

6. Modelos de variograma **6**
- 6.1. La lista `models` . . . 6
- 6.2. Registro: `add_model()` . . . 6
- 6.3. Compilación: `compile_model()` . . . 6

7. Predicción **6**
- 7.1. Recorrido de una llamada . . . 6
- 7.2. `prediction_setup()` . . . 7
- 7.3. `score_covariances()` . . . 7
- 7.4. Kriging simple y ordinario . . . 7
- 7.5. Método `"scores"`: `solve_kriging()` . . . 8
- 7.6. Método `"lambda"`: cokriging funcional . . . 8
- 7.7. `inner_product()` . . . 9
- 7.8. El objeto `SpatFDpred` . . . 9

8. Gráficos **9**

9. Validación cruzada: `leave_one_out()` **9**

10. Pruebas **10**

11. Cómo extender el paquete **10**
11.1. Agregar un modelo de variograma 10
11.2. Agregar un traductor (por ejemplo, `geoR`) 11
11.3. Agregar un método de predicción 11
11.4. Cambiar la estructura interna 11
11.5. Antes de cada *commit* 11

12. Limitaciones conocidas **11**

1. Principios de diseño

El paquete sigue la convención de los paquetes de modelado de R (`lm`, `predict`, `update`). Cinco reglas explican casi todas las decisiones del código:

1. **Los objetos son listas normales con clase.** No hay *environments* ni métodos `$` sobrescritos. R no copia los datos al copiar una lista mientras no se modifiquen, así que compartir referencias no ahorra memoria de forma apreciable.

2. **Nada se modifica en el sitio.** Toda función que cambia un objeto lo devuelve: `sfd <- add_model(sfd, ...)`.

3. **Lo funcional está oculto.** El usuario solo ve matrices y tablas. Las curvas suavizadas, los componentes principales, los datos originales y los modelos compilados viven en un único atributo, `internal`, al que el resto del código accede solo por medio de accesores (sección 4.3).

4. **El ajuste de variogramas está fuera del paquete.** SpatFD exporta los *scores* (`togstat()`), recibe los modelos ajustados (`toSpatFD()`, `add_model()`) y resuelve la predicción con código propio.

5. **Mensajes al usuario en inglés, comentarios en español.** Los mensajes de error y la documentación roxygen están en inglés; los comentarios del código, en español, con un título en mayúsculas al inicio de cada bloque (por ejemplo, extttCOMPILACIÓN:).

2. Flujo de trabajo

Paso	Función	Archivo	Resultado
1	`spatfd()`	`spatfd.R`	Objeto con una variable: suavizamiento + PCA funcional
2	`+`	`spatfd.R`	Objeto con varias variables
3	`togstat()`	`gstat.R`	Objeto `gstat` con un campo por *score*
4	(fuera del paquete)	—	Variogramas ajustados con `gstat`
5	`add_model()`	`add_model.R`	El objeto con el modelo compilado y registrado
6	`predict()`	`predict.R`	Objeto `SpatFDpred`: curvas y varianzas
7	`plot()`, `print()`	`methods-*.R`	Gráficos y resúmenes
8	`leave_one_out()`	`leave_one_out.R`	Objeto `SpatFDloo`: comparación de modelos y métodos

3. Organización de los archivos

Cada archivo agrupa una función exportada (o una familia) con sus funciones internas. Las funciones internas que usan varios archivos están en `utils.R`. La tabla lista todo lo que hay en `R/`.

Archivo	Exportado	Interno
`SpatFD-package.R`	Documentación del paquete	Todos los `@importFrom`
`spatfd.R`	`spatfd()`, `+.SpatFD`	—
`methods-SpatFD.R`	`print`, `summary`, `plot` de `SpatFD`; `var_names()`	`print.summary.SpatFD`, `plot_components()`
`gstat.R`	`togstat()`, `toSpatFD()` y sus métodos	—
`add_model.R`	`add_model()`	`compile_model()`
`models.R`	`models`, `availablemodels()`	`check_params()`
`predict.R`	`predict.SpatFD`	`predict_scores()`, `predict_lambda()`, `prediction_setup()`, `krige_scores()`
`kriging.R`	—	`score_covariances()`, `cross_dist()`, `solve_kriging()`, `constrained_solve()`
`methods-SpatFDpred.R`	`print`, `plot` de `SpatFDpred`	`predicted_fd()`, `observed_fd()`, `map_grid()`, `plot_curves()`, `plot_maps()`
`leave_one_out.R`	`leave_one_out()`, `print.SpatFDloo`	`curve_distance()`, `refit()`
`utils.R`	—	Constructor, accesores, `score_ids()`, `model_table()`, `models_message()`, `evaluate_curves()`, `inner_product()`
`data.R`	Documentación de `air_mexico`, `air_bogota`, `vowels`	—

Otras carpetas:

- `tests/testthat/`: una prueba por archivo de `R/` (sección 10).
- `dev/`: scripts de uso (`01_uso_paquete.R`) y de comparación con `gstat` (`02_comparacion_gstat.R`), y documentos técnicos. Está en `.Rbuildignore`.
- `archive/`: versiones anteriores y material de referencia. Está en `.gitignore`.
- `man/` y `NAMESPACE`: generados por roxygen. No se editan a mano; se regeneran con `devtools::document()`.

4. El objeto `SpatFD`

4.1. Estructura

```
sfd                                  lista, clase "SpatFD"
|-- $scores     matriz: un sitio por fila; coordenadas y luego <var>.PC1, <var>.PC2, ...
|-- $variance   data.frame: una fila por variable (basis, no. PC, variance)
|-- $models     data.frame: una fila por modelo registrado (tabla resumen)
`-- attr(, "internal")               lista
    |-- $coords     matriz con las coordenadas de todos los sitios
    |-- $variables  lista, una entrada por variable:
```

```
|   `-- $PM10
|       |-- $data   matriz original (tiempos x sitios)
|       |-- $fd     curvas suavizadas (objeto fd de fda)
|       |-- $pca    resultado de fda::pca.fd()
|       `-- $args   list(t, basis, nbasis, lambda): para repetir el análisis
`-- $models     lista, una entrada por modelo registrado:
    `-- $lmc
        |-- $table       data.frame del modelo (var, model, rango, sigma, alpha1)
        |-- $variograms  lista de funciones gamma(h), una por id de variograma
        `-- $sills       vector con la meseta total de cada id
```

4.2. Invariantes

Todo el código supone que se cumple lo siguiente. Cualquier función que cree o modifique un objeto debe mantenerlo.

- Las filas de `$scores` son todos los sitios del objeto, ordenados por nombre. Un sitio donde no se observó una variable tiene `NA` en sus *scores*.
- Las columnas de `$scores` son primero las coordenadas (las de `internal$coords`) y luego los *scores*, con nombres `<variable>.PC<k>`. El orden de las columnas importa: es el que recibe `gstat` (sección 5).
- Los *ids* de los variogramas son los nombres de los *scores* (`PM10.PC1`) para los directos, y dos de ellos unidos por un punto (`NO2.PC1.PM10.PC1`) para los cruzados, en cualquier orden.
- `$models` es siempre `model_table()` del objeto. Solo `new_spatfd()` y `set_models()` la escriben.
- `var_names()` devuelve las variables en orden alfabético. Las funciones que recorren variables usan ese orden.

4.3. Constructor, accesores y modificadores

Están en `utils.R`. El resto del paquete **nunca** usa `attr(x, "internal")` directamente; así, si la estructura interna cambia, solo cambia este archivo.

Función	Qué hace
`new_spatfd(scores, variance, coords, variables, models)`	Arma el objeto y calcula `$models`
`get_coords(sfd)`	Coordenadas de todos los sitios
`get_variable(sfd, v)`	Entrada completa de la variable: `data`, `fd`, `pca`, `args`
`get_fd(sfd, v)`, `get_pca(sfd, v)`	Curvas suavizadas y PCA de la variable
`get_models(sfd)`	Lista de modelos compilados
`set_pca(sfd, v, pca)`	Devuelve el objeto con el PCA de `v` reemplazado
`set_models(sfd, models)`	Devuelve el objeto con los modelos reemplazados y `$models` recalculada
`score_ids(sfd)`	Vector con la variable de cada *score*, nombrado con el id del *score*
`model_table(sfd)`	La tabla resumen de los modelos
`models_message(sfd)`	La tabla resumen como texto, para los mensajes de error

4.4. Construcción: `spatfd()`

1. Verifica los argumentos con `stopifnot()` y mensajes con nombre.
2. Suaviza: base B-splines o Fourier sobre `range(t)`, penalización `vec2Lfd()` y `smooth.basis()`.
3. Calcula el PCA con `pca.fd(fd, nharm)`. Si se da `var`, repite el PCA con el mínimo número de componentes que alcanza esa varianza.
4. Arma `scores`, `variance` y la entrada de la variable en `variables`, con `args` para poder repetir el análisis, y llama a `new_spatfd()`.

4.5. Combinación: `+.SpatFD`

- Exige que las coordenadas tengan las mismas columnas.
- Si una variable está en los dos objetos, conserva la del primero y avisa con un *warning*.
- Coordenadas: unión de los sitios, primero los del primer objeto.
- *Scores*: unión completa por sitio (*full join*) con `match()`. No se indexan filas de un *data frame* por nombre, porque R hace coincidencias parciales: el sitio `s1` tomaría los *scores* de `s10`. Hay una prueba para este caso.
- `variables`: concatenación de las listas.
- Modelos: los del primer objeto y luego los del segundo. Un nombre repetido se renombra en secuencia (`model 2`, ...) con un mensaje.

5. Intercambio con `gstat`

5.1. `togstat()`

Crea un objeto `gstat` con un campo por *score* (fórmula `score ~ 1`) y quita en cada campo los sitios sin observación. Los campos siguen el orden de las columnas de `$scores`. **Ese orden no es indiferente**: `gstat::fit.lmc()` llega a ajustes distintos según el orden de los campos. Hubo una regresión por ordenar alfabéticamente, y hay una prueba en `test-spatfd.R` que lo impide.

5.2. `toSpatFD()`: el esquema tabular

Todo modelo, venga de donde venga, se convierte a un *data frame* con una fila por estructura y las columnas:

Columna	Contenido
`var`	id del variograma (directo o cruzado)
`model`	nombre del modelo en la lista `models`
`rango`	parámetro de rango
`sigma`	meseta parcial (puede ser negativa en los cruzados)
`alpha1`	parámetro de forma (`kappa` en `gstat`)

`toSpatFD()` es genérica. Métodos actuales: `gstat` (resultado de `fit.lmc()`), `list` (lista nombrada de `variogramModel`, que se envuelve como un `gstat`) y `default` (error). La traducción de nombres de `gstat` está en `toSpatFD.gstat()`. La anisotropía se descarta.

6. Modelos de variograma

6.1. La lista `models`

`models` (exportada) es una lista nombrada de funciones `function(h, rango, sigma, alpha1, ...)` que devuelven $\gamma(h)$. Hay 18 espejos de `gstat`, que reproducen sus valores (prueba en `test-models.R`), y 4 modelos propios. Todas validan sus parámetros con `check_params()`, que define los dominios:

Dominio	Usado en
`'positive'` (por defecto)	`rango` y `alpha1` de casi todos
`'non-negative'`	`rango` de `linear` (0 = sin meseta)
`'up to 2'`	`rango` de `power`, `alpha1` de `stable`

`check_params()` solo revisa los parámetros que recibe: `nugget` no le pasa `rango`. El signo de `sigma` no se revisa.

6.2. Registro: `add_model()`

1. Si el modelo no viene en el esquema tabular, lo traduce con `toSpatFD()`.
2. Verifica que los ids sean *scores* del objeto o parejas de ellos, y que los modelos existan en `models`.
3. Compila con `compile_model()` y calcula las mesetas totales por id.
4. Guarda la entrada (`table`, `variograms`, `sills`) con `set_models()`. Un nombre repetido reemplaza el modelo sin cambiar su posición; sin nombre, se usa `next_model_name()`.

6.3. Compilación: `compile_model()`

Para cada id, cada fila de la tabla se convierte en un *closure* `function(h) do.call(model, c(list(h), args))`, y el variograma es la suma de esos *closures*. Los parámetros en `NA` se omiten, de modo que el modelo usa su valor por defecto. Cada estructura se evalúa una vez con `numeric(0)` al compilar: así un parámetro inválido falla al registrar el modelo y no al predecir.

7. Predicción

7.1. Recorrido de una llamada

```
predict.SpatFD(object, newcoords, model, variable, method, kriging, n)
 |-- si newcoords es sf/sfc:  map_grid()  -> grilla regular dentro del mapa
 |-- method = "scores":  predict_scores()
 |     `-- krige_scores()
 |           |-- prediction_setup()     verifica y prepara
 |           |-- score_covariances()    C(h) = meseta - gamma(h), por pareja de scores
 |           `-- solve_kriging()        resuelve el sistema (simple u ordinario)
 |-- method = "lambda":  predict_lambda()
 |     |-- prediction_setup()
 |     |-- score_covariances()
 |     `-- sistema propio (bloques por variable), con constrained_solve() si es ordinario
 `-- arma el objeto SpatFDpred
```

7.2. `prediction_setup()`

Es el paso común a los dos métodos. Escoge el modelo (si hay uno solo, no hace falta nombrarlo), elige las variables (por defecto, todas las que cubre el modelo), ordena las columnas de `newcoords` según las del objeto (por nombre, o por posición si no tienen nombre) y separa, para cada *score*, los sitios donde se observó y sus valores. Devuelve:

Elemento	Contenido
`model`, `compiled`	Nombre del modelo y su entrada compilada
`pcs`	Lista: los ids de los *scores* de cada variable
`sites`, `values`	Sitios y valores observados de cada *score*
`newcoords`	Coordenadas verificadas, con nombres de fila

Todos los errores sobre el modelo incluyen `models_message()`, la tabla de modelos del objeto.

7.3. `score_covariances()`

Devuelve una matriz-lista $q \times q$ (una fila y una columna por *score*) con la función de covarianza de cada pareja, $C(h) = C(0) - \gamma(h)$. Las parejas sin variograma cruzado quedan en `NULL`, es decir, independientes. Los modelos sin meseta (`unbounded_models`: `linear`, `power`, `logarithmic`, `spline`, `legendre`) solo se admiten con kriging ordinario, con la pseudo-covarianza $C(h) = -\gamma(h)$ (sección 7.4); con kriging simple dan un error.

7.4. Kriging simple y ordinario

`predict()` y `leave_one_out()` reciben `kriging`, por defecto `"ordinary"`. El simple supone media conocida e igual a cero (los *scores* están centrados). El ordinario supone una media constante desconocida y agrega una restricción por campo: los pesos del campo que se predice suman 1 y los de cada uno de los demás suman 0 (cokriging ordinario). `constrained_solve()` arma y resuelve, por LU, el sistema

$$\begin{pmatrix} \Sigma & F \\ F^T & 0 \end{pmatrix} \begin{pmatrix} W \\ M \end{pmatrix} = \begin{pmatrix} C_0 \\ E \end{pmatrix}, \qquad \sigma^2 = C(0) - \textstyle\sum_{\text{columna}} [C_0; E] \circ [W; M],$$

donde F marca las filas de cada campo y E el campo objetivo de cada columna. Dos consecuencias de las restricciones guían el diseño:

- **Las constantes se cancelan.** Sumar una constante a un bloque de covarianzas no cambia ni los pesos ni los multiplicadores ni la varianza. Por eso los modelos sin meseta funcionan con $C(h) = -\gamma(h)$, y los que tienen meseta conservan $C(h) = C(0) - \gamma(h)$, que está mejor condicionada.
- **Da igual centrar las curvas.** En lambda, como los pesos de la variable objetivo suman 1 y los de las demás 0, las medias se cancelan: $\hat{\chi}^r = \sum_p \sum_i \lambda_i^{rp} \chi_{s_i}^p$ con las curvas sin centrar. El código sigue sumando la media a las curvas centradas, que es la misma cuenta y vale también para el simple.

La matriz del ordinario no es definida positiva, así que no se usa Cholesky. Las dos variantes se comparan con `gstat` (con y sin `beta = 0`) en `test-predict.R`, `test-ordinary.R` y `dev/02_comparacion_gstat.R`.

7.5. Método `"scores"`: `solve_kriging()`

Resuelve el kriging de q campos escalares (los *scores*). El detalle matemático está en `dev/diseno_solver.pdf`; las decisiones de implementación son:

- **Grupos.** Los campos conectados, directa o indirectamente, por covarianzas cruzadas forman un grupo, y cada grupo se resuelve por separado. Sin cruzados hay un kriging por *score*; con un modelo lineal de corregionalización completo hay un solo sistema de cokriging.
- **Distancias.** Se calculan una vez por conjunto distinto de sitios (los *scores* de una variable comparten sitios) y se guardan en caché.
- **Simple: un Cholesky por grupo.** Con $\Sigma = R^T R$ y $V = R^{-T} C_0$, la predicción es $V^T(R^{-T} z)$ y la varianza $C(0) - \sum V^2$ por columna, sin calcular los pesos. Si Cholesky falla, la matriz no es definida positiva: hay sitios duplicados o el modelo no es válido.
- **Ordinario: un LU por grupo** con `constrained_solve()`, con una restricción por campo del grupo.
- **Distancia cero exacta.** `cross_dist()` calcula la distancia coordenada a coordenada para que la de un sitio consigo mismo sea exactamente 0. De eso depende el efecto pepita.

`predict_scores()` reconstruye después cada curva sobre los coeficientes de la base: coefs $= \Xi \hat{f} +$ coefs(μ), con Ξ los coeficientes de las funciones propias.

7.6. Método `"lambda"`: cokriging funcional

`predict_lambda()` implementa el predictor de Bohórquez, Giraldo y Mateu (2017), con las curvas centradas:

$$\hat{\chi}^r_{s_0}(t) = \mu^r(t) + \sum_p \sum_i \lambda_i^{rp} \big(\chi^p_{s_i}(t) - \mu^p(t)\big).$$

Pasos:

1. **Intervalo común** T: la intersección de los intervalos de las variables. Si no coinciden, avisa con un mensaje; si no se cruzan, da un error.
2. **Matrices** $G^{pq}_{kl} = \int_T \xi^p_k \xi^q_l \, dt$. Para una variable observada en todo T, G^{pp} es exactamente la identidad (no se calcula). En los demás casos se usa `inner_product()`.
3. **Covarianza entre curvas** de cada pareja de variables, $\Omega_{pq}(h) = \sum_{k,l} G^{pq}_{kl} \, C_{pk,ql}(h)$, como función de h. Es `NULL` si no hay ningún término.
4. **Sistema** con un bloque por pareja de variables. Simple: $\Sigma\lambda = C_0$ con un Cholesky. Ordinario: `constrained_solve()` con una restricción por variable. Para cada variable objetivo solo cambian C_0 y la restricción.
5. **Curvas**: se evalúan en los tiempos de la variable dentro de T. Esa matriz es exactamente la predicción y es la que recibe el usuario.
6. **Varianza**: $\Omega_{rr}(0) - C_0^T \lambda$ (menos el multiplicador en el ordinario), con las mesetas del modelo. No puede ser negativa.
7. **Versión funcional** (solo para graficar): la matriz se proyecta por mínimos cuadrados en la base de la variable, o en B-splines sobre T si el intervalo es más corto.

Este método no usa `solve_kriging()`: el sistema es pequeño (un bloque por variable, no por *score*) y escribirlo aparte resultó más claro que generalizar el *solver* a valores matriciales.

7.7. `inner_product()`

`fda::inprod()` se detiene en un error relativo de 10^{-4}, que no basta para un sistema que se verifica a 10^{-8}. `inner_product()` usa la matriz exacta de productos internos de la base (`eval.penalty`) cuando las dos curvas están en la misma base y se integra en todo su intervalo. Si no, integra por Gauss-Legendre de 5 puntos en 1000 tramos (error de 10^{-15} con Fourier y 10^{-9} con B-splines en las pruebas).

7.8. El objeto `SpatFDpred`

```
pred                                    lista, clase "SpatFDpred"
|-- $PM10
|   |-- $pred   matriz: tiempos x sitios nuevos (curvas evaluadas)
|   `-- $var    vector: varianza de predicción por sitio nuevo
|-- attr(, "newcoords")   coordenadas de los sitios nuevos
|-- attr(, "method")      "scores" o "lambda"
|-- attr(, "crossed")     si se usó alguna covarianza cruzada
|-- attr(, "map")         el mapa sf, si se predijo sobre uno
`-- attr(, "internal")
    |-- $pred       curvas predichas como fd, por variable
    |-- $observed   curvas observadas como fd, por variable
    `-- $coords     coordenadas de los sitios observados
```

Contrato de los métodos: `predict_scores()` y `predict_lambda()` devuelven `list(curves, newcoords, crossed)`, donde `curves` tiene por variable `fd`, `pred` y `var`. Un método nuevo solo tiene que cumplir ese contrato y agregarse al `switch` de `predict.SpatFD`.

8. Gráficos

Función	Qué dibuja
`plot.SpatFD, mode = "functional"`	Curvas suavizadas, con `plot.fd` de `fda`
`plot.SpatFD, mode = "pca"`	Todos los componentes en una página (`plot_components()`), con `plot.pca.fd` componente a componente
`plot.SpatFDpred, mode = "curves"`	Observadas en gris y predichas en color (`plot_curves()`)
`plot.SpatFDpred, mode = "map"`	Predicción en un instante y varianza, lado a lado (`plot_maps()`)
`availablemodels()`	Los modelos de variograma en una grilla de 5×5

Política de `ask`. Ningún gráfico del paquete pide `<Enter>`. Cada uno guarda `par(ask = FALSE)` al empezar y restaura el estado del dispositivo al salir con `on.exit(..., add = TRUE)`. La razón: `fda::plot.pca.fd()` dibuja una página por componente y activa `ask`; si el dispositivo queda en ese modo, R pide `<Enter>` antes de cada gráfico nuevo, aunque sea el único. La única excepción es `plot(sfd, multiple = TRUE)`, que el usuario pide explícitamente. La prueba `test-plots.R` deja el dispositivo en modo `ask` y comprueba con el *hook* `before.plot.new` que ninguna página nueva se abre en ese modo.

Mapas. `map_grid()` toma cerca de `n` puntos regulares dentro del mapa con `sf::st_sample(type = "regular")`. `plot_maps()` reconstruye la grilla para `image()` a partir del paso mínimo entre coordenadas y arma la página con `layout()`: título, subtítulo, dos barras de color y dos mapas.

9. Validación cruzada: `leave_one_out()`

Para cada sitio de la variable:

1. `refit()` repite el suavizamiento y el PCA de la variable sin el sitio, con los mismos `args` y el mismo número de componentes. Las otras variables se reconstruyen una sola vez, sin quitar nada.

2. Las funciones propias nuevas toman el signo de las originales (por el signo de su producto interno). El signo de una componente es arbitrario, pero los variogramas cruzados del modelo dependen de él.

3. Se registran de nuevo los modelos con su tabla original: los variogramas **no** se vuelven a ajustar.

4. Se predice el sitio con cada combinación de modelo y método, y se mide el error con `curve_distance()`: la norma L^2 de la diferencia, exacta si las curvas comparten base y por `inner_product()` si no (por ejemplo, lambda sobre un intervalo común más corto).

Devuelve un `SpatFDloo`: `summary` (una fila por combinación, ordenada por error medio), `errors` (sitios × combinaciones) y `pred` (por combinación, matriz tiempos × sitios; `NA` fuera del intervalo en que se predijo).

10. Pruebas

Se corren con `devtools::test()`. Cada archivo usa datos simulados pequeños (10 sitios, 50 tiempos) y `gstat` como referencia externa siempre que existe.

Archivo	Qué verifica
`test-spatfd.R`	Construcción, `+` (incluido el caso `s1`/`s10`), orden de `togstat()`, que solo se expongan `scores`, `variance` y `models`
`test-models.R`	Los 18 espejos contra `gstat::variogramLine()`, estructuras anidadas, dominios de los parámetros, `toSpatFD()`
`test-predict.R`	Kriging y cokriging de *scores* contra `gstat`; lambda univariado contra las fórmulas escritas a mano; cokriging funcional contra el sistema por bloques; intervalos distintos; mapas; validación de argumentos; registro de modelos
`test-leave_one_out.R`	Cada error es el de repetir el análisis sin el sitio; diseños; modelos multivariados; intervalos distintos
`test-ordinary.R`	Kriging y cokriging ordinario contra `gstat`; modelos sin meseta; lambda ordinario independiente de centrar, contra el sistema con restricciones; `leave_one_out()` ordinario por defecto
`test-plots.R`	Ningún gráfico pide `<Enter>`; los componentes van en una página; `plot()` devuelve el objeto de forma invisible

Las pruebas pueden usar las funciones internas (`testthat` carga el paquete con `load_all()`). Además, `dev/02_comparacion_gstat.R` repite las comparaciones con `gstat` sobre los datos reales de México y se detiene si alguna diferencia relativa supera 10^{-8}.

11. Cómo extender el paquete

11.1. Agregar un modelo de variograma

1. Agregar la función a la lista `models` en `models.R`, con `check_params()` al inicio y `h = 0` resuelto explícitamente si la fórmula da `NaN` ahí.

2. Si tiene equivalente en `gstat`, agregar la traducción en `toSpatFD.gstat()` y el caso en la prueba de espejos.

3. Si no tiene meseta, agregarlo a `unbounded_models` en `kriging.R`: solo se podrá usar con kriging ordinario.

4. Actualizar la documentación de `models`.

11.2. Agregar un traductor (por ejemplo, `geoR`)

Escribir un método `toSpatFD.<clase>()` en `gstat.R` (o en un archivo propio) que devuelva el esquema tabular. `add_model()` lo usa sin cambios.

11.3. Agregar un método de predicción

Escribir `predict_<nombre>()` con el contrato de la sección anterior, agregarlo al `switch` y a `match.arg()` de `predict.SpatFD`, y documentarlo en `@details` y `@section Theory`. `leave_one_out()` lo admite si se agrega al diseño por defecto.

11.4. Cambiar la estructura interna

Solo hay que tocar `utils.R` (constructor y accesores) y las pruebas de estructura de `test-spatfd.R`. Si un cambio obliga a tocar otros archivos, probablemente se está saltando un accesor.

11.5. Antes de cada *commit*

```
devtools::document()     # NAMESPACE y man/
devtools::test()
source("dev/02_comparacion_gstat.R")
devtools::check()        # 0 errores y 0 warnings
```

12. Limitaciones conocidas

- Media constante: el kriging ordinario estima una media constante; todavía no hay kriging universal (media con tendencia espacial).
- Vecindario global y distancias euclidianas: sirve para coordenadas proyectadas y un número moderado de sitios.
- El método lambda con varias variables supone que los `t` de `spatfd()` están en una escala común. En los datos de México (`t = 1, 2, ...`) eso equivale a suponer que PM10 y NO2 empiezan a la misma hora.
- Solo se traducen modelos de `gstat`, sin anisotropía.
- Falta migrar desde la versión CRAN el muestreo óptimo, la clasificación y la simulación.